HiViewshed: An Interactive Online Viewshed Analysis System for Multiple Observers

Ye Wu*
College of Electronic Science, National University of Defense Technology
Changsha, China
yewugfkd@nudt.edu.cn

Mengyu Ma
College of Electronic Science, National University of Defense Technology
Changsha, China
mamengyunudt@gmail.com

Luo Chen
College of Electronic Science, National University of Defense Technology
Changsha, China
luochen@nudt.edu.cn

ABSTRACT

Viewshed analysis is a fundamental operation of most geographic information system software. The last decade has witnessed an explosion in the availability of terrain data with high resolutions. Hence, it is essential to develop efficient viewshed analysis system to process these data. This paper demonstrates the design and engineering of an interactive online viewshed analysis system named HiViewshed on massive grid terrains. HiViewshed takes advantage of hybrid parallel computing architectures with message-based and shared-memory parallelism, and it has overcome problems and deficiencies of the existing systems. The main features of the system include (a) MPI/OpenMP based parallel computing of total viewshed analysis for multiple observation points in a large region, (b) parallel DEM data access with multi-resolution strategy ensuring online analysis quality with limited bandwidth and computing resources, and (c) favorable parallel acceleration and scalability in cluster architecture. Experimental results have shown that, it can respond in an acceptable time with several hundreds of observation points simultaneously.

CCS CONCEPTS

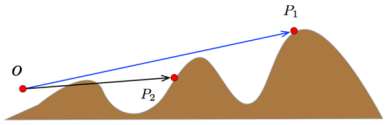
• Information system • Information systems applications • Spatial-temporal systems • Geographic information systems

KEYWORDS

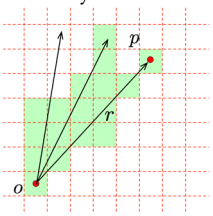
Parallel computing, Viewshed analysis, Grid, Hybrid parallelism

1 Introduction

A viewshed is an area that is visible on a terrain surface from one or more specific locations. Viewshed analysis uses the elevation value of each cell of the digital elevation model (DEM) to find the areas visible to a give location O within a given radius of interest. The location O can also have a relative altitude. The general analysis has received extensive attention in spatial information applications and location-based services in the fields of communication, planning, environmental protection, tourism, and public security. For example, it can be used to set a radar station or a TV transmitter station, to calculate a maximum integrated coverage of signals for a given group of wireless communication base stations, to assist with maritime navigation, and so on.



(a) Illustration of visibility



(b) Raster representation of visibility

Figure 1: Visibility and Its Raster Representation.

Suppose O denotes an observe point, r denotes the radius of interest, and P denotes a terrain cell. A target P is visible from observation O if and only if the distance of p from O is, at most, r and all the elevations along the line of sight from O to P are below OP. For example, in Figure 1(a), P_1 is visible from O, while P_2 is not. A DEM consists of a set of elevation samples and can be represented as a raster or as a vector-based triangular irregular network. The most commonly used form of DEM is raster with uniform grids [1]. So the following of the paper mainly focuses on the discussion of terrain with regular grids. For points in a two-dimensional projected plane, the viewshed of O is a set of terrain cells that can be seen from O within distance r (see Figure 1(b)). Formally,

$$vs(O) = \{p | dist(O, p) \leq r \text{ and } p \text{ is visible from } O\}$$

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
CSAE 2019, October 22–24, 2019, Sanya, China
© 2019 Association for Computing Machinery.
ACM ISBN 978-1-4503-6294-8/19/10$15.00
https://doi.org/10.1145/3331453.3361327

The guideline viewshed algorithm on grid terrains is R3 [2] introduced by Franklin and Ray and has great accuracy. It uses the digital differential analyzer to determine which cells the line of sight intersects. Existing improved approaches focus on the calculation and optimization of visible regions starting from a single observation point or a few observation points. The accelerating techniques can be classified into two categories: parallel-based optimization and approximation-based computation. The former one optimizes the single observation point access analysis algorithm using a dedicated hardware architecture (such as multi-core CPU, GPU) [3-5]; or uses a distributed computing strategy, by splitting the input area and assigning calculation tasks to different processing units of a multiprocessor system [6,7]. The second category of algorithms takes advantage of approximating techniques [8, 9], such as generalization, sampling, interpolation, etc. The above method is effective in dealing with small and medium-sized problems under single-user conditions. However, in the context of high concurrency, it is difficult to carry out the simultaneous calculation of visible areas of large-area with multi-observation points and support online interactive services.

On the other hand, there are some online or near real-time GIS frameworks. Esri offers real-time capabilities via ArcGIS [10] extension for the server, which is deployed on the cloud. Moreover, systems for processing large-scale spatial data such as GeoSpark [11] and Simba [12] are based on Apache Spark cluster. However, these systems mainly focus on the online processing of structured text, map tile or feature data, not designed for raster data analysis.

Since viewshed analysis is quite compute-intensive, the increase in the volume of high-resolution terrain data makes it urgent to develop interactive systems. Furthermore, geographic information systems are often used to visualize results for the end- users to help to make decisions. Most of the time, to be useful in practice, acquiring fast and accurate results in a reasonably short time are very important. In order to support the online analysis of large-area multi-observation points in the Web environment, we have designed and implemented a hybrid parallel-based online analysis system HiViewshed. The system, which supports simultaneous online computing of large-scale observation points, is based on high-performance architectures and distributed spatial databases. It realizes multi-resolution data organization and parallel access to terrain data in spatial databases and supports multiple observations viewshed analysis in a large area. The system greatly shortens the overall calculation time and effectively guarantees the interactive quality of online viewing calculation under the constraints of limited bandwidth and computing resources. Overall, the goal of the system is to deliver a high performance, scalable and efficient viewshed analysis system capable of delivering near real-time results for a relatively large number of observations. Meanwhile, to process tasks exceed the number of processor cores and large scale datasets that not fit in memory, HiViewshed introduces a message-based task pipeline to optimize parallel execution for optimal performance.

The rest of the paper is organized as follows: Section 2 presents an overview of our system framework. The main online service, task-scheduling engine and parallel viewshed analysis module are also described in detail. In Section 3, we summarize the experimental results and give performance comparisons. Then we conclude the paper in Section 4.

2 HiViewshed Framework Overview

To begin with, we mainly focus on the hybrid parallel framework to improve existing viewshed calculation algorithms, and our main purpose is fast interaction and visualization of the viewshed analysis results.

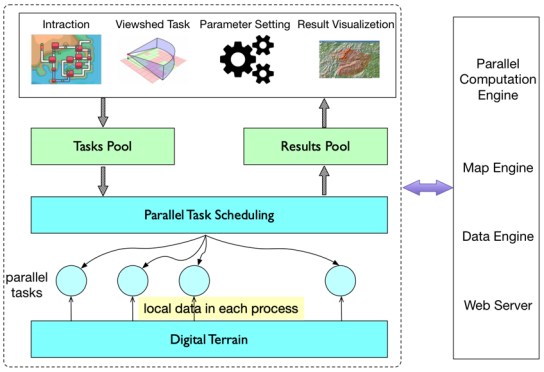


Figure 2: Architecture of HiViewshed.

As depicted in Figure 2, HiViewshed contains three main components: (a) *Online Service Framework*, (b) *Task Scheduling Engine* and (c) *Parallel Viewshed Analysis*. The system is built based on the parallel computation engine, map engine, data engine, and the webserver. The four fundamental modules provide necessary environmental support for the system. When getting a request from the front web browser, the task scheduling engine pipelines the request into the tasks pool in an FIFO mode. The parallel task scheduling launches different tasks to viewshed analysis modules simultaneously. As time consumption of different tasks varies, the scheduling module will allocate new tasks to the idle process to realize dynamic scheduling. After pushing the calculation results into the results tool, the map engine will wake up to render and visualize results, and send them to the client. Since the aim of our system is to answer viewshed analysis in near real-time, the architecture of HiViewshed must guarantee response times. The three functional modules are described in detail as follows.

2.1 Online Service Framework

The online service framework encapsulates the HiViewshed framework as a REST service, making it suitable to be called by various web-browser-based applications. The requests, which are wrapped up in HTTP requests, contain a list of tuples $\langle center, height, radius \rangle$ to generate the viewshed of every observer. Particularly, *center* denotes the latitude and longitude coordinate of an observer, which is encoded with GeoHash [9] to

offer arbitrary precision and reduce the size of the URL. For example, the latitude and longitude coordinates of *28.9881819154N*, *119.426426244E* falls within the GeoHash bounding box of "*wthuvt5smkv8*" with error limited to 1.8cm.

The framework receives the call parameters through the REST service interface, then encapsulates the call parameters into task messages through parameter parsing, sends them to the tasks pool, and then starts the parallel view analysis module. When the analysis results are presented, the framework filters the results corresponding to the request from the results pool, aggregates and sends them to the client user through the REST service interface.

2.2 Task Scheduling Engine

The task scheduling engine listens to the front client's requests and sends the results back to the results pool when a task is completed. The process is asynchronous as the engine enqueues the requests and only when any computing executor is idle it dequeues a request.

Especially, through the subscription/publishing framework, the task scheduling engine receives the analysis task sent by the application end to the tasks pool through the online service framework, analyzes the composition and characteristics of the task, and decomposes the task into a series of subtasks.

Every subtask is described as a key-value pair. Figure 3 shows a representation of the task description. The key consists of priority, session id, user id, observer longitude and latitude coordinate, and timestamp to uniquely indicate a record. In addition, the value field is composed of height, radius, group id, and map level to help to complete computing.

key				
Priority	Session ID	User ID	Observer LngLat	TimeStamp
value				
Height		Radius	Group ID	Map Level

Figure 3: Key-Value Design for Task Scheduling Engine.

The subscription/publishing framework is an important component for parameter/result transfer between the online service framework and the parallel view analysis module, as well as control synchronization. When there is an idle processor, the engine will take out a subtask from the tasks pool and send it to the analysis module. Each node adopts a parallel strategy to read the corresponding terrain data from the digital terrain database and performs a viewshed analysis calculation based on a specially designed algorithm. All the results are cached in the results pool until a task is completed.

2.3 Parallel Viewshed Analysis

The system calculates the visible area based on the LOS (Line of Sight) maximum slope method. The basic steps are: taking a ray from the observation point to the target point, determining the grid point on the ray in the DEM data by the Bresenham algorithm, and sequentially calculating the LOS slope from the observation point to the grid point. The maximum slope is recorded and continuously updated when traversing each point on the ray. The visibility of a point is determined by comparing the LOS slope of the current point to the current maximum slope. Our parallel framework can also easily be extended to other viewshed analysis methods.

2.3.1 Multi-observer Viewshed Calculation based on Hybrid Parallel Model. The parallel viewshed analysis module uses a two-level parallel model, i.e., MPI+OpenMP hybrid parallel mode to accelerate the multi-observer viewshed computation. Since the viewshed calculations of different observation points are not dependent on each other, tasks have good parallelism.

MPI program takes the calculation of a single observation point as a task unit. Most of the processes are used to compute the subtasks and just one process manages the tasks. When the system receives a computation request, it initializes a static load balancing strategy. Specifically, it first estimates the complexity of the task by counting the number of target points involved in the task unit and getting the resolution of the DEM data. Second, it sorts all the tasks by its complexity to ensure that the amount of computation allocated to each process is approximately equal. After the task started for a while, when a process is idle, it will dynamically fetch a subtask from the task-scheduling engine until the tasks pool is empty.

To calculate the viewshed of a single observation point, we need to ingest the DEM data of the adjacent area near the observation point. OpenMP is suitable to accelerate the computation based on shared memory. We divide the region of interest into several small regions by equal angle division around the observer. Then, each region is assigned to an OpenMP process.

2.3.2 Multi-scale Parallel Strategy to Access Large-scale Terrain Data. Computing viewshed directly with the high-precision DEM data can achieve higher precision. However, the calculation complexity is proportional to the size of DEM data. In practice, it is also difficult to return the calculation result in real time when there are many requests. Due to the limitations of the client display resolution, it is not necessary to display highly accurate results when exploring low-resolution maps. As a result, the system realizes a multi-scale parallel strategy to access large-scale terrain data, which can reduce the calculation amount of the viewshed analysis without affecting the visualization effect while coping with a large amount of viewshed calculation requests.

The multi-scale DEM data access service uses the pyramid data model to organize DEM data. The l_0 layer of the pyramid stores the DEM data of the original resolution, and the pixel values of other layers are compressed by sampling methods such as bilinear sampling. The data resolution of level l_k is 1/2 of that of level l_{k-1}. We use the strategy illustrated in [7] to map cells and compute elevation for each level of the data.

When multiple front clients concurrently invoke the viewshed analysis service, the display resolution R is also included in the requests. Before each task is pushed into the tasks pool, the multi-scale DEM data access service is called in parallel. The DEM data access service determines the level L of the required DEM data for each task according to the display resolution R. Formally, the calculation formula is as follows.

$$L = min(\lceil log_2(R/r_0) \rceil, l_{max})$$

where r_0 is the resolution of the l_0 layer, and l_{max} is the highest level of the DEM data.

Next, the DEM data access service determines the required DEM data space range according to the user-defined viewshed analysis radius. Only the corresponding small pieces of DEM data on the calculated level L are returned to different processes for calculation. Each process uses the returned DEM data of different resolutions to get their respective analysis tasks.

3 Experiments

We implemented the algorithm in C++ using OpenMP and MPI. The algorithm was compiled with g++ and optimization level-O3. We use a big 30-m-resolution SRTM of China, with a size of 72001×48001. We conduct all the experiments on an SMP (Symmetrical Multi-Processing) server, which has an Intel Xeon E5-4620 2 CPU (8-core, 2.6GHz per core), 192GB memory and 40 TB raid 6-disk cluster. All experiments are conducted based on our HiGIS [10] system.

The client only requires a standard browser that supports HTML5, including Chrome, Firefox, IE, and more.

3.1 Use Case

This subsection describes a use case to illustrate the viewshed analysis effects offered by the system.

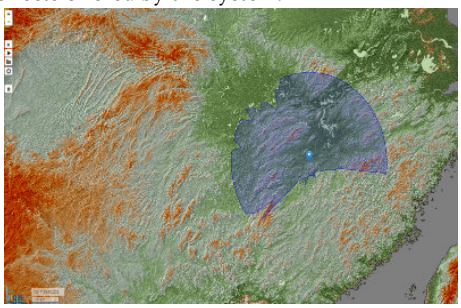


Figure 4: Online Viewshed Analysis for a Single Observation Point.

Select the "Create Observation Point" sub-function and click on the map to locate an observation point at the selected location. Then select the "Calculate viewshed" sub-function to perform the online calculation of the visible area centered on the observation point. The user can drag the observation point arbitrarily, and the system will update the visible area in real time according to the position of the observation point after the movement, as shown in Figure. 5.

To demonstrate the performance of several hundreds of observation points viewshed analysis, the system supports bulk-generated arbitrary observation points. Select the "Viewshed Analysis in Bulk" sub-function to perform the online parallel calculation of the visible area centered on each observation point. The user can drag any of the observation points arbitrarily, and the system will update the visible area in real time according to the position of the observation point after the movement. At the same time, the user can set parameters for any observation point, including the height of the observation point, the radius of the viewshed, and the color of the rendering area. After the setting is completed, the system calculates the updated result in real time according to the parameters. Figure 6 shows the viewshed result of randomly generated 512 observation points.

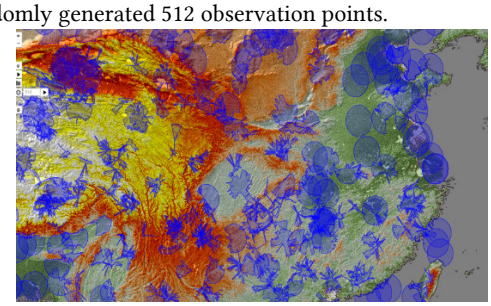


Figure 5: Online Parallel Viewshed Analysis for Multiple Observation Points (512 Points).

3.2 Performance Evaluation

In order to carefully investigate the viewshed analysis system performance, we discuss the efficiency varying different radius, the number of observation points and the number of processes.

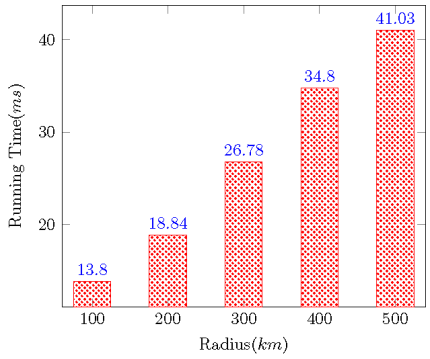


Figure 6: Average Running Time Varying Radius.

Exp1: Impact of the radius. In this experiment, we fix the number of observers to 500 and the number of processes to 32. We demonstrate the average running time of an observer varying viewshed radius, where

$$average\ time = \frac{\text{total running time}}{\text{num of observers}}.$$

The size of data we have to process is square proportion to the radius. As shown in Figure 7, with the increase of radius, the average running time cost increases. On the other hand, due to our hybrid parallel strategy, the execution time exhibits a sub-

linear correlation with the radius. For example, the execution time of radius $300km$ is 2 times (not square proportion 3^2=9 times) more than of a $100km$ radius. Especially, dynamic fetching of subtasks of each process ensures that almost no process is idle during the task is running.

Exp2: Effect of the number of processes. In this experiment, the radius is fixed to $2km$ and the number of observations is set to 500. To demonstrate the scalability of HiViewshed, we evaluate its performance with different process settings. We vary the number of total processes to be 1, 2,4,8,16,32 and 64, respectively. As depicted in Figure 8, the running time cost of HiViewshed decreases as we increase the number of processes. That makes sense because more threads can handle more tasks in parallel. However, more tasks mean more resource conflicts. While the number of threads exceeds 16, which is just equal to the number of server cores, the performance declined instead.

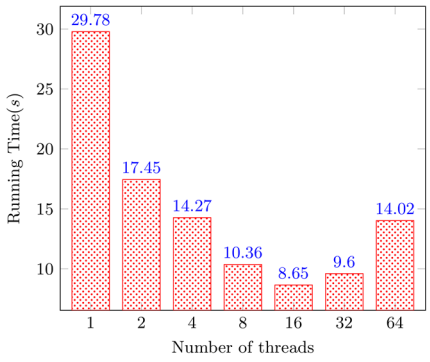


Figure 7: Total Running Time Varying Number of Threads.

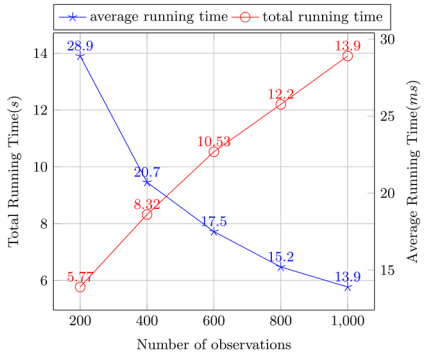


Figure 8: Running time Varying Number of Observations.

Exp3: Scale with the number of observations. In this experiment, we fix the radius to $2km$ and the number of processes to 16. The total running time and average running time are shown in Figure 9, represented by hollow circle and star, respectively. It can be seen that there is a clear positive correlation between the number of observations and the total running time, which is what we expected. However, the average running time spent on each observer shows a completely different trend. There are two reasons for this phenomenon. First, the dynamic task scheduling strategy makes full use of the idle processors, and then the time spent on each task is reduced. Second, with the help of OpenMP on each processor, the system can handle more than one task on one processor once a time.

4 Conclusions

Due to the large dataset size, viewshed analysis for multiple observers may take a long time to complete. In this paper, we presented a system called HiViewshed, an interactive viewshed analysis system for large-scale spatial data. The system combines task scheduling strategy and OpenMP/MPI technology to accelerate bulk viewshed analysis. Experimental results show that HiViewshed can return user requests in an acceptably short time.

In the future, we would like to scale up our system to even larger data. We would also like to explore a dynamic task scheduling strategy for the multi-user environment.

ACKNOWLEDGMENTS

This work is supported in part by National Natural Science Foundation of China (Grant No. 41871284) and National Natural Science Foundation of Hunan Province (Grant No. 2019JJ50718).

REFERENCES

[1] A. Prescott and L. Toma (2018). A Multiresolution approach for viewsheds on 2D terrains. In Proceedings of the 26th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems. ACM, New York, NY, USA, 63-72.
[2] W. R Franklin, and C. Ray (1994). Higher isn't necessarily better: Visibility algorithms and experiments. In Advances in GIS research: sixth international symposium on spatial data handling. Taylor & Francis Edinburgh. 751-770.
[3] C. R. Ferreira, M. V. A. Andrade, S. V. G. Magalhães, W. R. Franklin, and G. C. Pena (2014). A parallel algorithm for viewshed computation on grid terrains. Journal of information and data management. 5(2), 171-180
[4] A. Osterman, L. Benedicic, and P. Ritosa (2014). An I/O-efficient parallel implementation of an R2 viewshed algorithm for large terrain maps on a CUDA GPU. International Journal of Geographic Information Science. 28(11), 2304--2327.
[5] Y. L. Zhao, A. Padmanabhan, and S. W. Wang (2013). A parallel computing approach to viewshed analysis of large terrain data using graphics processing units. International Journal of Geographic. Information Science. 27(2), 363-384.
[6] T. X. Yu, L. Y. Xiong, M. Cao (2016). A new algorithm based on region partitioning for filtering candidate viewpoints of a multiple viewshed. International Journal of Geographical Information Science. 30(11). 2171-2187.
[7] W. Dou, Y. Li, Y. Wang (2018). A Fine-granularity scheduling for parallel XDraw viewshed analysis. Earth Science Informatics, 11: 433-447.
[8] S. Tabik, A. R. Cervilla, E. Zapata, and L. F. Romero (2015). Efficient data structure and highly scalable algorithm for total viewshed computation. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing. 8(1), 304-310.
[9] M. V. Kreveld (1996). Variations on Sweep Algorithms: efficient computation of extended viewsheds and class intervals. In Proc. Symposium of Spatial Data Handling. 15–27.
[10] ArcGIS Online (2019). www.arcgis.com.
[11] J. Yu, J. Wu, and M. Sarwat (2015). Geospark: a Cluster Computing Framework for Processing Large-scale Spatial Data. In Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems, Article 70.
[12] D. Xie, F. Li, B. Yao, G. Li, L. Zhou, and M. Guo (2016). Simba: Efficient In-memory Spatial Analytics. In Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference, 1071–1085